

Disease Prediction Assistant

(MediSense AI — Intelligent Disease Prediction Platform)

A PROJECT REPORT

Submitted by

Vansh Chopra

Registration No: 12406635

*in partial fulfillment for the award of the degree
of*

BACHELOR OF TECHNOLOGY

School of Computer Science and Engineering



L OVELY
P ROFESSIONAL
U NIVERSITY

Lovely Professional University, Punjab

July 2026

TABLE OF CONTENTS

Sr No	Topic	Page
1	Abstract	3
2	Introduction	4
3	Problem Statement	5
4	Objectives	6
5	Scope	7
6	System Design & Analysis	8
7	Technologies Used	9
8	Architecture Diagram	10
9	Module Description	11
10	Implementation	11
11	Code Snippets	12-13
12	Screenshots of Web Pages	14-17
13	Testing	18
14	Results	19
15	Output Screenshot	19
16	Conclusion	20
17	Future Work	20
18	References	21

ABSTRACT

MediSense AI (submitted here as “Disease Prediction Assistant”) is a solo-developed, single-page Streamlit web application built to provide fast, AI-assisted preliminary disease prediction from user-described symptoms. Rather than relying on a fixed symptom checklist, the platform lets a user describe how they are feeling in natural conversational language, then combines a cloud large-language model with three independently trained machine learning classifiers to estimate the most likely underlying condition.

The prediction engine draws on a dataset spanning 754 diseases and 377 distinct symptoms, and offers a choice of K-Nearest Neighbours, Decision Tree, or Logistic Regression as the inference algorithm, with Logistic Regression achieving the platform's headline accuracy of 86.7%. Each prediction returns not just a single verdict but a ranked differential diagnosis, a plain-language clinical explanation generated by the integrated Google Gemini model, a suggested specialist referral, and general lifestyle recommendations, alongside a clear medical disclaimer.

Beyond the core prediction workflow, MediSense AI includes an AI-driven conversational consultation module for structured symptom collection, a prediction history and comparison view, automated report generation, and a real-time analytics dashboard tracking model usage, confidence trends, and the most frequently detected symptoms. The entire system — authentication, prediction logic, data storage, and visualisation — is implemented as a single Streamlit application, demonstrating how a modern Python data-science stack can be used to build a complete, deployable clinical-assistant tool without a separate backend service.

INTRODUCTION

Access to fast, reliable, preliminary medical guidance remains a challenge for many people. When symptoms first appear, individuals often turn to generic web searches that return inconsistent or overwhelming information, or they delay seeking care altogether because they are unsure whether a visit to a doctor is warranted. A structured, AI-assisted tool that can interpret symptoms described in plain language and offer a statistically grounded, explainable prediction can help bridge this gap — not as a replacement for a doctor, but as a first, informed step.

MediSense AI is designed as exactly this kind of assistant. It allows a user to either fill in a structured prediction form or describe their symptoms conversationally through an AI consultation chat, after which the system extracts the relevant clinical variables and runs them through a trained machine learning model. Unlike purely large-language-model-based symptom checkers, which can be difficult to verify and are prone to inconsistent reasoning, MediSense AI grounds its primary diagnosis in classical, interpretable ML classifiers (KNN, Decision Tree, and Logistic Regression) trained on a labelled disease-symptom dataset, while still using a cloud LLM (Google Gemini) to make the interaction feel natural and to explain the result in accessible language.

The application is built entirely in Python using the Streamlit framework, which allows the user interface, prediction logic, and data storage to live in a single deployable application rather than a separate frontend and backend. This project demonstrates the practical use of Streamlit, scikit-learn, and a cloud generative-AI API together to build a responsive, end-to-end healthcare-assistant platform, complete with authentication, a live dashboard, an analytics suite, and downloadable reports.

PROBLEM STATEMENT

People experiencing new or unfamiliar symptoms frequently lack an accessible, trustworthy first point of reference before deciding whether professional medical attention is needed. Existing digital symptom checkers tend to fall into one of two categories: rigid, form-based systems that require exact symptom selection from long checklists and offer little explanation for their output, or purely conversational AI tools that can describe plausible-sounding conditions without any grounding in a validated statistical model, making their reliability difficult to assess.

MediSense AI addresses this gap by combining both approaches. It allows natural-language symptom entry through an AI consultation chat while ensuring that the actual diagnosis is produced by a trained, evaluable machine learning classifier rather than free-form language generation alone. The system further supports the user by presenting a ranked differential diagnosis instead of a single answer, showing the confidence associated with each prediction, and clearly recommending when a specialist should be consulted — all while being transparent that its output is a statistical estimation, not a medical diagnosis.

OBJECTIVES

The main objectives of the Disease Prediction Assistant (MediSense AI) are:

1. To allow users to describe symptoms in natural, conversational language and have them automatically parsed into structured clinical variables using an NLP pipeline.
2. To provide statistical disease prediction using multiple trained machine learning models — K-Nearest Neighbours, Decision Tree, and Logistic Regression — with a visible confidence score for each prediction.
3. To combine ML-based prediction with cloud LLM-based clinical reasoning (Google Gemini) so predictions are explained to the user in plain, understandable language.
4. To present a ranked differential diagnosis of the most probable conditions rather than a single rigid verdict.
5. To offer specialist referral suggestions and general lifestyle or care recommendations alongside every prediction.
6. To maintain a history of past predictions and consultations, and allow the user to compare results across models.
7. To provide a real-time analytics dashboard that tracks model usage, average confidence, and the most frequently detected symptoms.
8. To allow users to generate a downloadable, formal report summarising a consultation or prediction session.
9. To build the complete system — authentication, prediction engine, storage, and analytics — as a single, easily deployable Streamlit application.

SCOPE

The current scope of MediSense AI covers user registration and secure login (including a separate administrator login), an AI-driven consultation module for natural-language symptom collection, a configurable prediction engine offering a choice of three trained machine learning models, a differential-diagnosis and clinical-reasoning results view, prediction history with comparison support, downloadable report generation, and a live analytics dashboard summarising platform usage and diagnostic trends. The application runs as a single, self-contained Streamlit app, making it lightweight to deploy and easy to demonstrate end-to-end.

The project also leaves clear room for future expansion. The prediction dataset, currently covering 754 diseases and 377 symptoms, can be extended with more recent and regionally relevant clinical data; the cloud LLM integration can be supplemented with a fully offline local model for privacy-sensitive deployments; and the platform can be extended toward mobile access, multilingual symptom input, and integration with real electronic health record (EHR) systems. Overall, the project demonstrates how a modern Python data-science and generative-AI stack can be combined into a single practical tool that supports faster, better-informed health-related decisions.

SYSTEM DESIGN & ANALYSIS

MediSense AI is designed as a single-tier Streamlit application in which the user interface, prediction logic, model inference, and data persistence all run within one Python process, removing the need for a separate frontend/backend split. The application begins with user authentication, where a patient can register or sign in, or an administrator can access a separate admin login flow.

Once authenticated, the user lands on a central Dashboard that summarises recent activity — total predictions, consultations, average confidence, and live system status (AI consultation availability, loaded ML models, database connectivity, and NLP pipeline status). From the Dashboard, the user can start an AI Consultation, where symptoms are collected conversationally and structured in real time on a visible clinical-status panel (current stage, consultation progress, clinical confidence, and detected symptom count).

The structured symptom data feeds into the Prediction Engine (“Predict” module), where the user selects an inference algorithm — KNN, Decision Tree, or Logistic Regression — and executes inference. The engine returns a primary diagnosis with a confidence score, a ranked differential-diagnosis list, a specialist referral, recommended actions, and a natural-language clinical explanation generated via the integrated Gemini model, together with an execution timeline documenting each processing step (NLP extraction, model execution, and confidence scoring).

Every prediction and consultation is persisted so that the History and Compare modules can retrieve past sessions, and the Analytics module aggregates this stored data into live charts — model usage distribution, confidence trends, symptom frequency, and a multi-model performance radar. The overall design favours transparency: at each stage the user can see exactly which model produced a result, how confident it was, and why the recommendation was made, while a persistent medical disclaimer reinforces that outputs are statistical estimations rather than a professional diagnosis.

TECHNOLOGIES USED

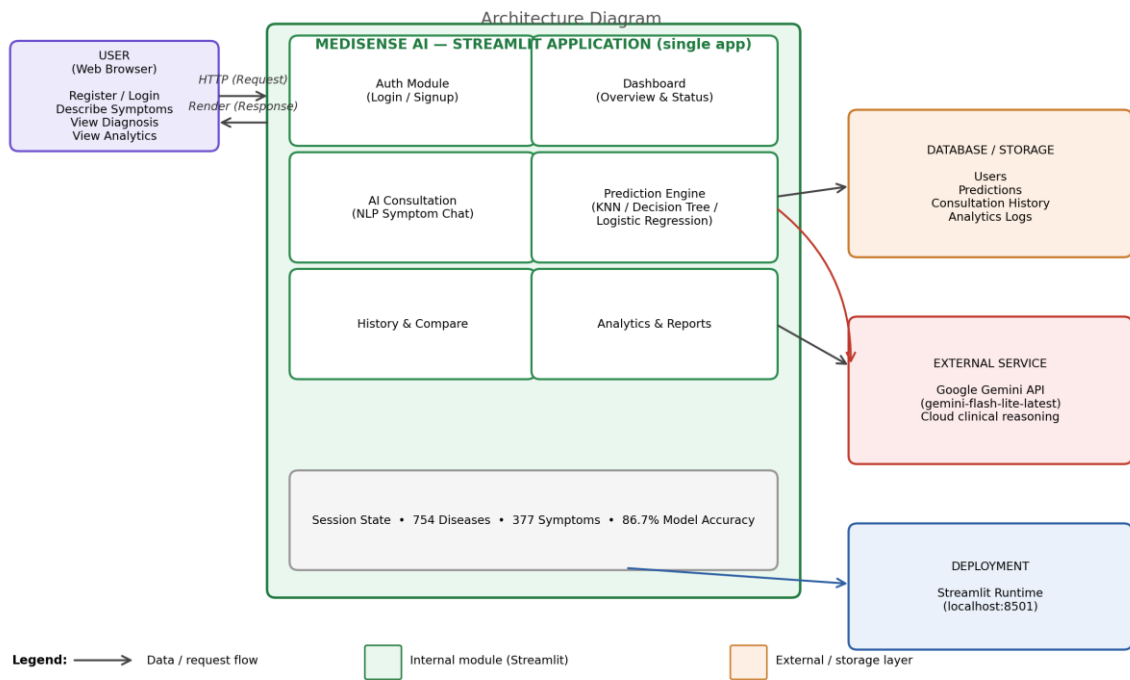
MediSense AI is built entirely on a Python data-science stack, combining a lightweight web framework, classical machine learning, and a cloud generative-AI model within a single application. The table below summarises each core technology and its role in the platform.

Technology	Purpose
Streamlit	Single Python framework used to build the entire application — UI, routing, session state, and backend logic — as one deployable app.
Python 3	Core programming language for application logic, data processing, and model inference.
scikit-learn	Provides the K-Nearest Neighbours, Decision Tree, and Logistic Regression classifiers used by the Prediction Engine.
Pandas / NumPy	Handle the disease-symptom dataset (754 diseases, 377 symptoms) and numerical computation during inference.
Google Gemini API (gemini-flash-lite-latest)	Cloud generative-AI model used in the AI Consultation module for natural-language symptom collection and clinical-reasoning explanations.
NLP Pipeline	Extracts structured clinical variables (symptoms, duration, severity) from the user's free-text description during consultation.
Plotly	Renders the interactive analytics visualisations — confidence gauge, model-usage donut chart, symptom-frequency bars, and performance radar.
SQLite / Local Database	Stores user accounts, prediction records, consultation history, and analytics logs.
Streamlit Session State	Manages authentication state and in-session data, including a Local AI fallback mode when the cloud model is unavailable.

Together, these technologies let MediSense AI deliver structured, explainable disease prediction alongside a natural conversational interface, without requiring a separate backend service or client framework.

ARCHITECTURE DIAGRAM

MediSense AI — Disease Prediction Assistant



MODULE DESCRIPTION

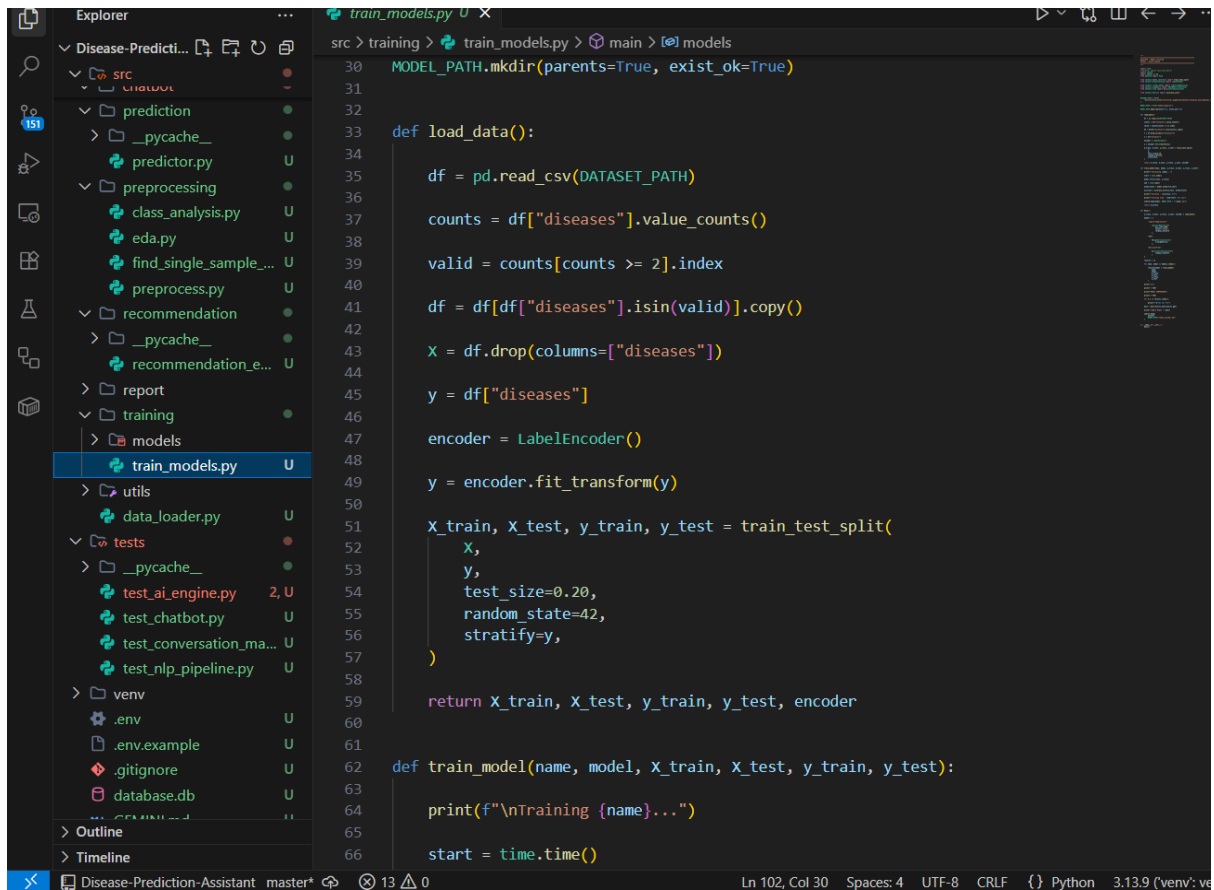
- **Authentication Module:** Handles patient registration, patient login, and a separate administrator login flow to access the secure clinical workspace.
- **Dashboard Module:** A central command centre showing total predictions, consultations, average confidence, current AI system status, and quick actions to reach every other module.
- **AI Consultation Module:** A conversational interface where users describe symptoms naturally; a live clinical-status panel tracks consultation stage, progress, confidence, and detected symptoms as the conversation proceeds.
- **Prediction Engine (Predict) Module:** Runs the selected machine learning model (KNN, Decision Tree, or Logistic Regression) on the collected symptoms and returns a primary diagnosis, differential diagnosis, confidence score, and AI-generated clinical reasoning.
- **History & Compare Module:** Stores every past prediction and consultation, and allows the user to review or compare results across sessions and models.
- **Analytics & Reports Module:** Aggregates stored predictions into real-time charts (model usage, confidence trends, symptom frequency, and model performance) and allows an official report to be generated for a completed consultation.

IMPLEMENTATION

MediSense AI is implemented as a single Streamlit application written in Python. The frontend and backend logic share the same process: Streamlit renders each module (Dashboard, Consult, Predict, History, Compare, Report, Analytics, About) as a page within one app, while session state manages authentication and in-progress consultation data. The three prediction models are trained using scikit-learn and loaded at runtime by the Prediction Engine, which also logs an execution timeline (NLP extraction, model execution, and confidence scoring) for transparency. The AI Consultation module calls the Google Gemini API (gemini-flash-lite-latest) for natural-language understanding and reasoning, with a Local AI fallback mode available when the cloud model is not reachable. All prediction and consultation records are persisted to a local database, which in turn powers the History, Compare, and Analytics modules, and interactive charts are rendered using Plotly. The complete application is served locally via the Streamlit runtime for development and testing.

CODE SNIPPETS

The following pages are reserved for source-code screenshots from the MediSense AI codebase. Replace each placeholder box below with a screenshot of the corresponding code from your editor (e.g. VS Code) once ready — recommended snippets are suggested under each box.



```
30 MODEL_PATH.mkdir(parents=True, exist_ok=True)
31
32
33 def load_data():
34
35     df = pd.read_csv(DATASET_PATH)
36
37     counts = df["diseases"].value_counts()
38
39     valid = counts[counts >= 2].index
40
41     df = df[df["diseases"].isin(valid)].copy()
42
43     X = df.drop(columns=["diseases"])
44
45     y = df["diseases"]
46
47     encoder = LabelEncoder()
48
49     y = encoder.fit_transform(y)
50
51     X_train, X_test, y_train, y_test = train_test_split(
52         X,
53         y,
54         test_size=0.20,
55         random_state=42,
56         stratify=y,
57     )
58
59     return X_train, X_test, y_train, y_test, encoder
60
61
62 def train_model(name, model, X_train, X_test, y_train, y_test):
63
64     print(f"\nTraining {name}...")
65
66     start = time.time()
```

Fig 1. Load Models

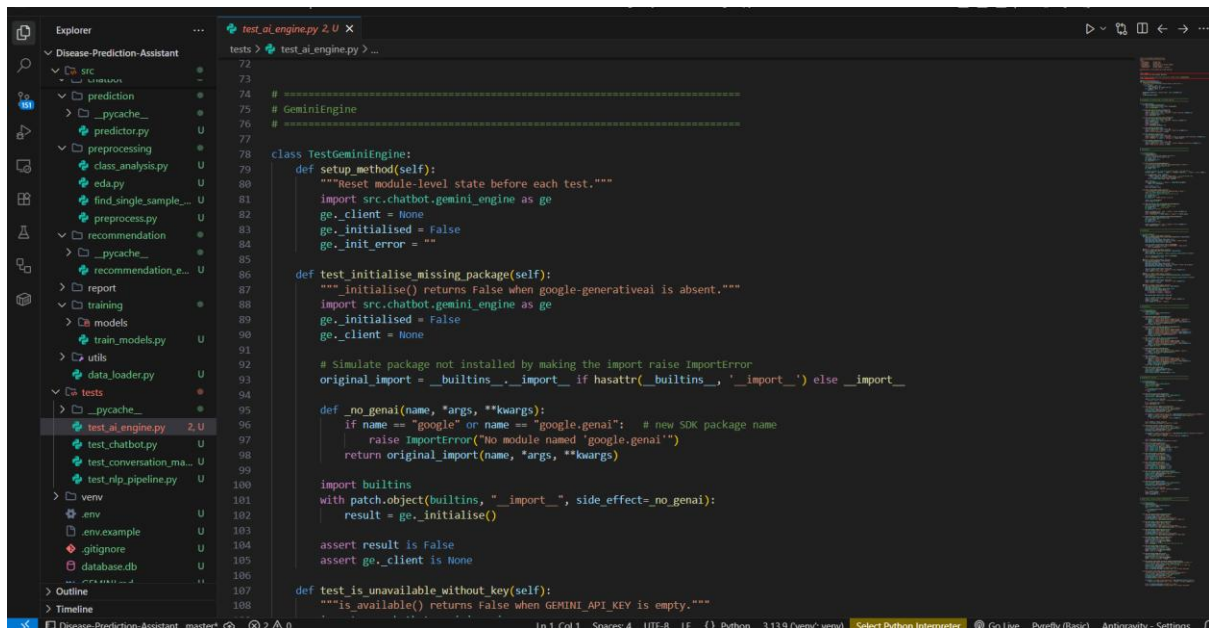


Fig.2. Testing AI Engine

```

1 from datetime import datetime
2
3 # pyrefly: ignore [missing-import]
4 import streamlit as st
5 import plotly.graph_objects as go
6
7 from app.utils.navigation import ANALYTICS, CONSULTATION, PREDICTION, REPORT
8 from app.utils.ui import escape_html, render_html
9 from app.components.logs_modal import render_logs_modal
10
11
12 def go_to(page_id: str) -> None:
13     st.session_state.current_page = page_id
14     st.rerun()
15
16
17 def render_dashboard() -> None:
18     """Renders the Premium AI SaaS Dashboard."""
19
20     user = st.session_state.get("user", {"name": "guest"})
21     now = datetime.now()
22     greeting = "Good morning" if now.hour < 12 else "Good afternoon" if now.hour < 18 else "Good evening"
23     display_name = escape_html(user.get("name", "Guest").split(" ")[0])
24     ai_engine = st.session_state.get("ai_engine_label", "Standard Consultation")
25
26     current_time = now.strftime("%I:%M %p")
27     current_date = now.strftime("%A, %b %d %Y")
28
29     # Get stats
30     predictions = st.session_state.get("prediction_history", [])
31     predictions_made = len(predictions)
32     consultations_today = 1 if predictions_made > 0 else 0 # Simulated for now
33     avg_conf = 88.5 # Simulated average confidence
34
35     last_disease = escape_html(predictions[-1]["disease"]) if predictions else "None"
36     last_conf = f"{predictions[-1]['confidence']:.1f}%" if predictions else "0%"
37     last_sys = "Respiratory" if last_disease in ["Pneumonia", "COVID-19", "Asthma"] else "Cardiovascular" if last_disease in ["Heart Disease"]

```

Fig.3. Dashboard logic

```

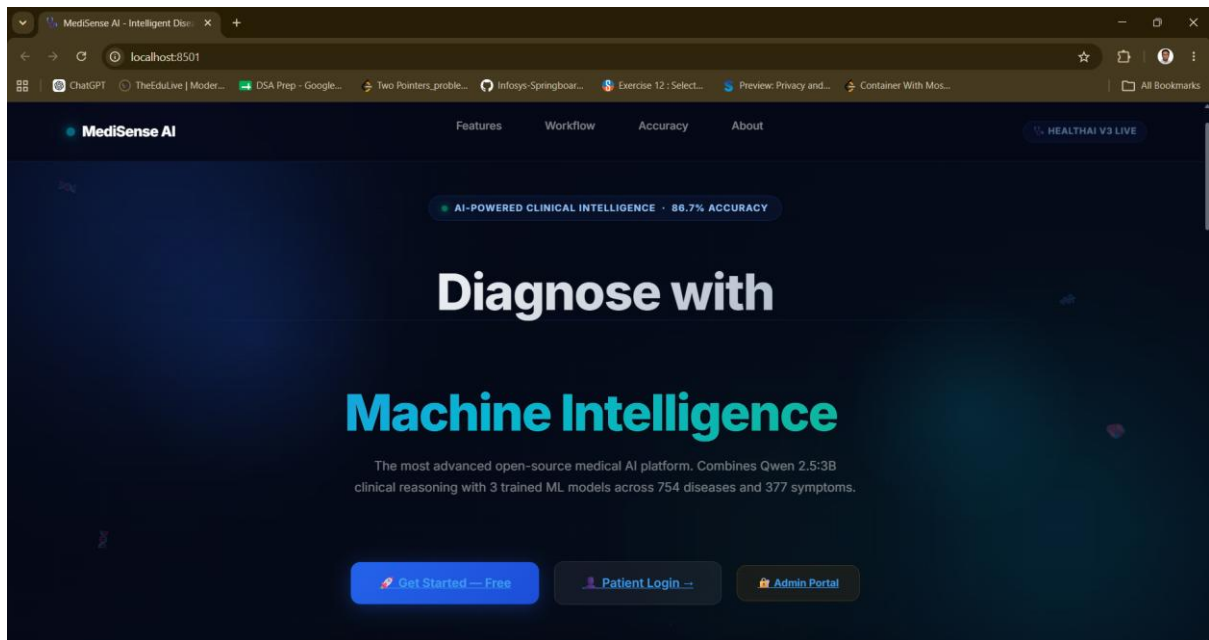
122 def render_prediction() -> None:
123     model_choice = st.selectbox(
124         "Prediction Algorithm",
125         ["Logistic Regression (Recommended)", "Decision Tree", "KNN"],
126         label_visibility="visible"
127     )
128     st.session_state.selected_model = model_choice
129
130     with c2:
131         # Align button
132         render_html("<div style='height: 1.7rem;'></div>")
133         predict_clicked = st.button("Execute Inference", use_container_width=True)
134
135     if predict_clicked:
136         with st.spinner(f"Running inference on {st.session_state.selected_model}..."):
137             result, error = run_prediction_bridge(symptoms, st.session_state.selected_model)
138
139             if error or not result:
140                 st.error(error or "Prediction completed but returned no disease categorization.")
141                 return
142
143             top_disease_name = result["disease"]
144             top_disease_conf = result["confidence"]
145
146             # Analytics Tracking
147             if "analytics" in st.session_state:
148                 st.session_state.analytics["total_predictions"] += 1
149                 base_model = st.session_state.selected_model.replace(" (Recommended)", "")
150                 if base_model in st.session_state.analytics["model_usage"]:
151                     st.session_state.analytics["model_usage"][base_model] += 1
152                 for sym in symptoms:
153                     st.session_state.analytics["symptoms_detected"][sym] = st.session_state.analytics["symptoms_detected"].get(sym, 0) + 1
154                     st.session_state.analytics["predicted_diseases"][top_disease_name] = st.session_state.analytics["predicted_diseases"].get(top_disease_name, 0) + 1
155
156             if "prediction_history" not in st.session_state:
157                 st.session_state.prediction_history = []
158
159             st.session_state.prediction_history.append({

```

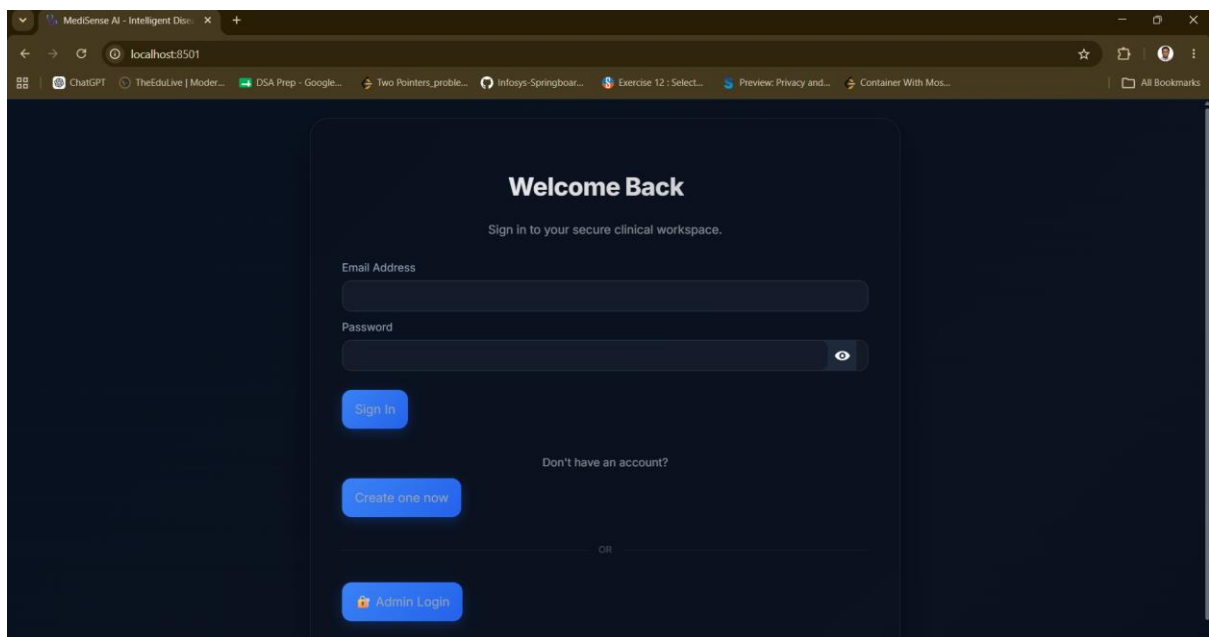
Fig 4. Prediction Logic

SCREENSHOTS OF WEB PAGES

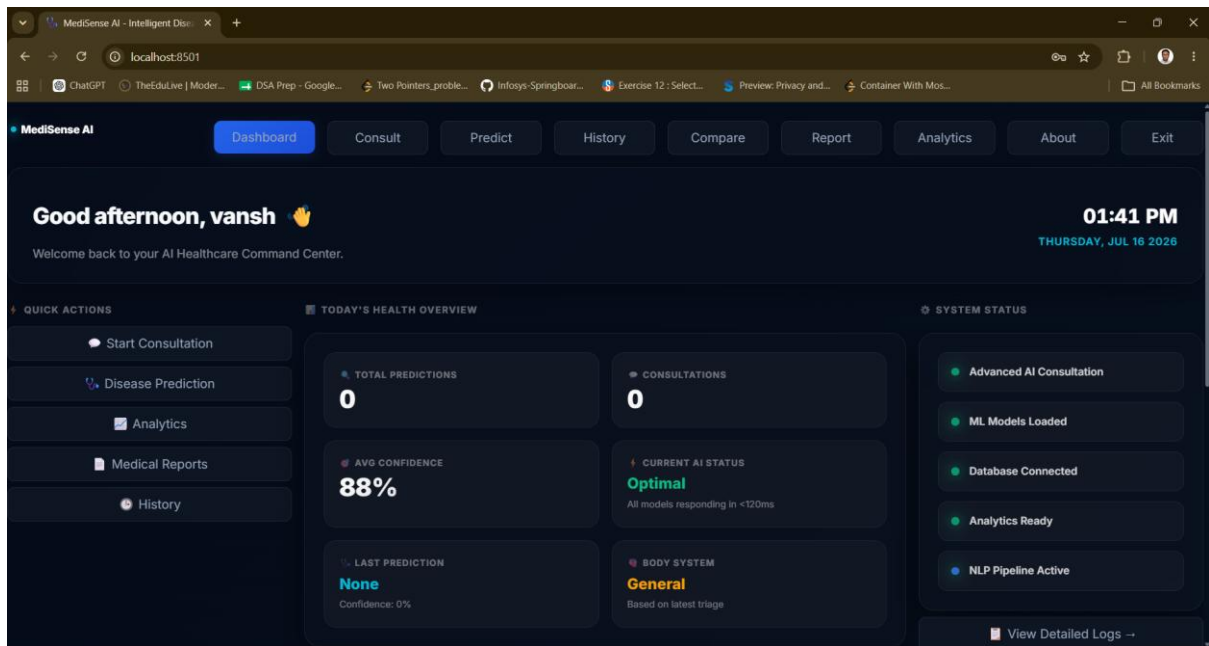
1. Home Page



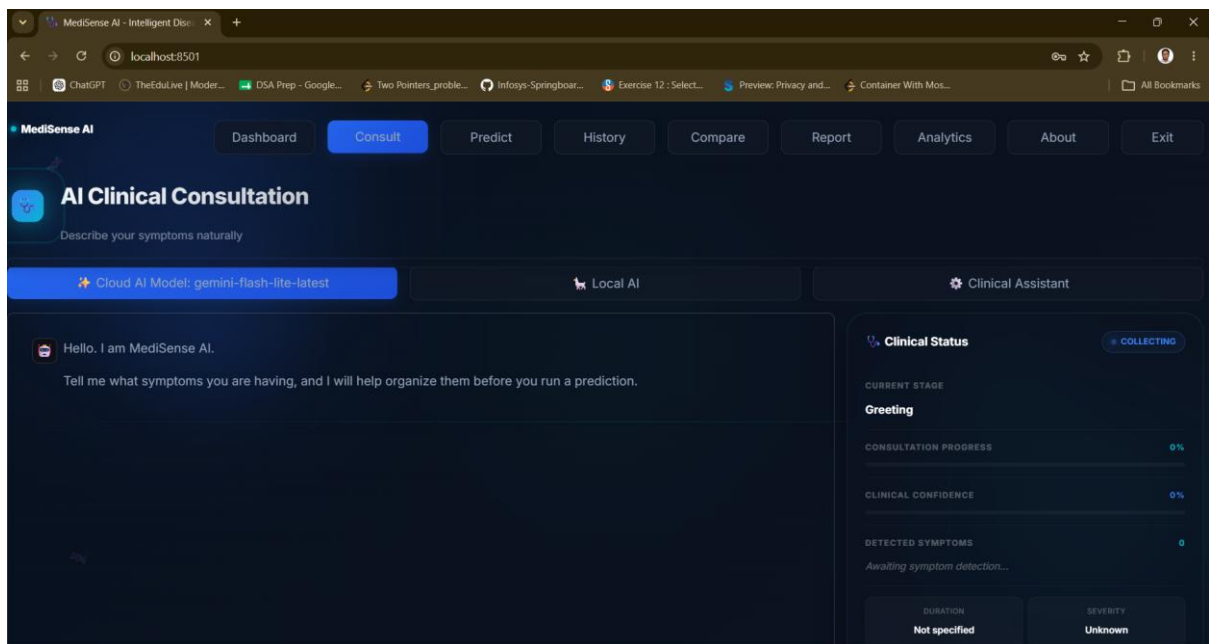
2. Login Page



3. Dashboard



4. AI Consultation



5. Prediction Engine — Diagnosis Result

MediSense AI

DashboardConsultPredictHistoryCompareReportAnalyticsAboutExit

 **Diagnostic Engine**

Statistical inference across KNN, Decision Tree, and Logistic Regression models.

Prediction Algorithm

Logistic Regression (Recommended)

Execute Inference

PRIMARY DIAGNOSIS MATRIX

hypokalemia

LOWGENERAL

37.9%

DIFFERENTIAL ANALYSIS

hypokalemia	37.9%
urinary tract infection	7.8%
labyrinthitis	6.6%
injury to the arm	4.9%
tietze syndrome	4.0%

SYSTEM EXECUTION TIMELINE

13:47:46.681

Extracted 12 clinical variables via NLP

13:47:46.681

Executed Logistic Regression (Recommended) Algorithm

13:47:46.681

Diagnosis isolated with 37.9% confidence

NLU CLINICAL REASONING

CLASSIFIER: LOGISTIC REGRESSION (RECOMMENDED)

The Machine Learning model predicted that the patient might have hypokalemia based on the symptoms provided. Hypokalemia refers to low potassium levels in the blood, which can cause various symptoms including nausea, vomiting, abdominal pain, and muscle weakness or cramps.

Here's how some of the symptoms align with hypokalemia:

- **Nausea:** Low potassium levels can affect your digestive system.
- **Vomiting:** Similar to nausea, low potassium might lead to stomach upset.
- **Ankle stiffness or tightness, arm stiffness or tightness, hip stiffness or tightness:** These could be due to muscle weakness caused by hypokalemia. Potassium is important for proper muscle function.
- **Loss of sensation and feeling cold:** Hypokalemia can affect nerve function, leading to numbness and a sense of coldness.
- **Hand or finger pain:** Muscle cramps and spasms are common with low potassium levels.
- **Heavy menstrual flow:** While not directly related to hypokalemia, it could be due to hormonal imbalances that might accompany this condition.

It's important to note that these symptoms can also be associated with other conditions. The Machine Learning model is a tool designed to provide insights based on patterns in data.

 SPECIALIST REFERRAL

General Physician

 RECOMMENDED ACTIONS

Rest and stay hydrated.

 LIFESTYLE ADJUSTMENTS

Maintain a healthy diet and monitor your symptoms.

 **MEDICAL DISCLAIMER**

This telemetry dashboard generates diagnostic estimations based on predictive statistical algorithms. It does not replace professional clinical consultations. Always contact your healthcare provider with any medical questions.

Generate Official Report

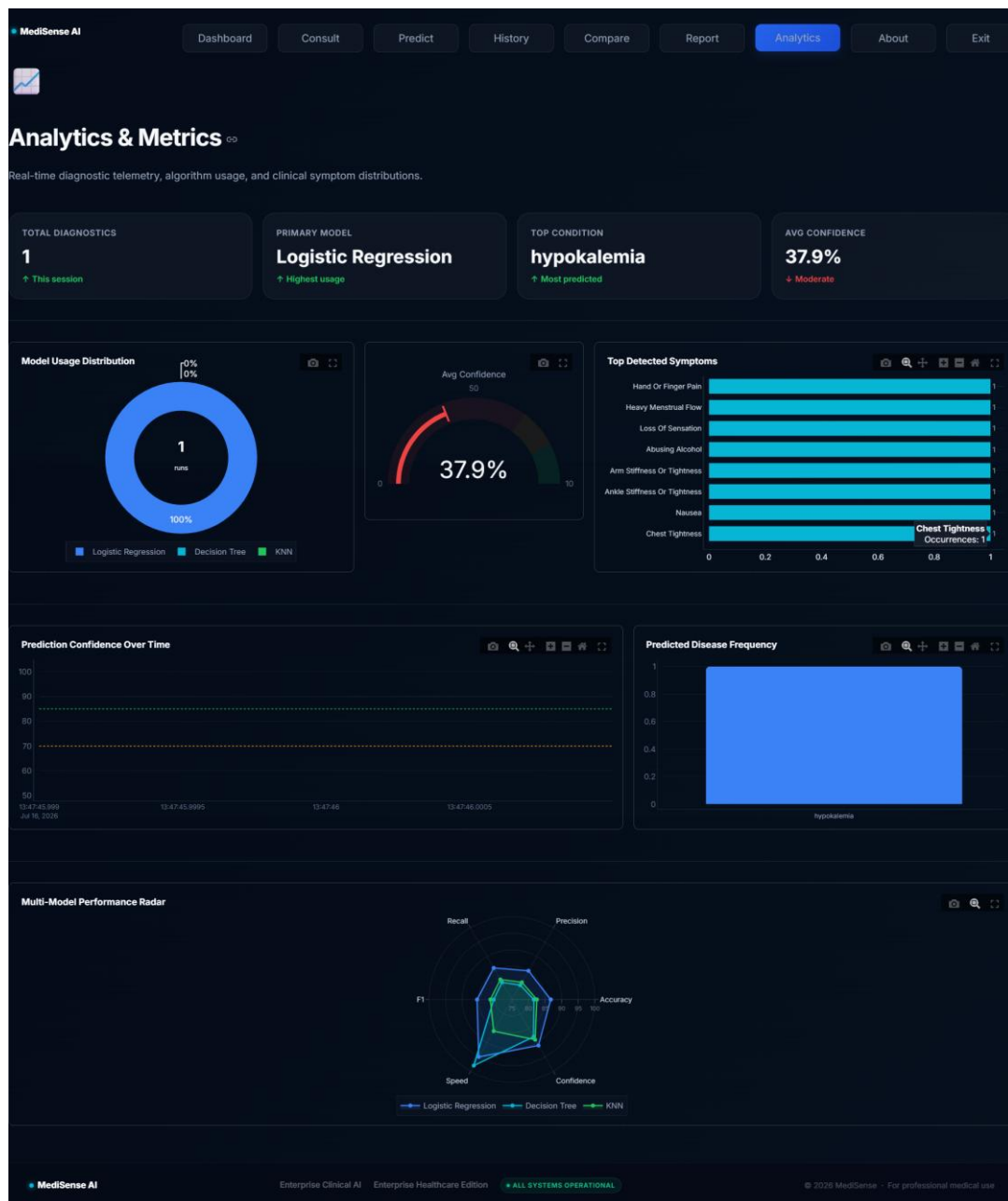
MediSense AI

Enterprise Clinical AIEnterprise Healthcare Edition

ALL SYSTEMS OPERATIONAL

© 2026 MediSense - For professional medical use

6. Analytics & Metrics



TESTING

MediSense AI was tested to confirm that authentication, the AI consultation flow, disease prediction across all three models, and the analytics dashboard function correctly. Functional testing was performed on each major module, and the results are summarised below.

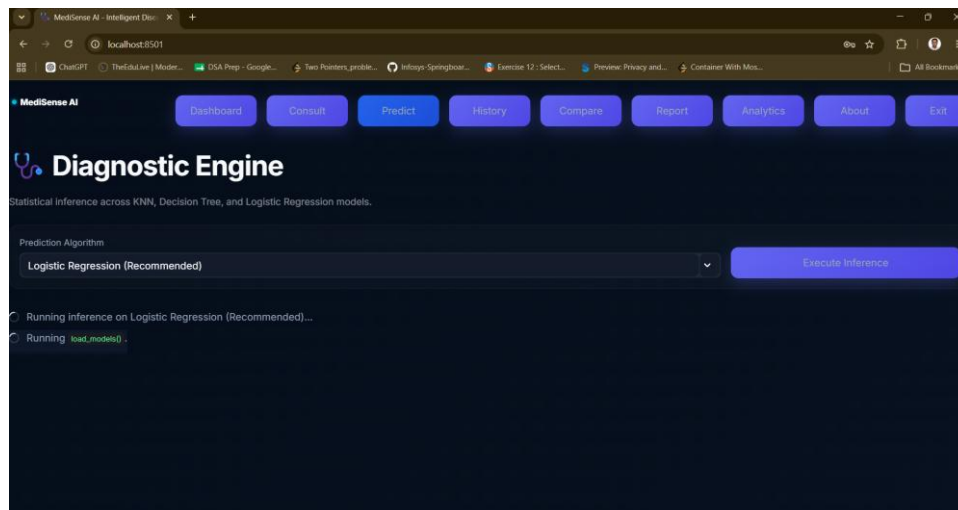
Test Case	Expected Result	Status
Patient Registration & Login	New account is created and the user can sign in successfully	Passed
Admin Login	Administrator can access the secure admin workspace via a separate login flow	Passed
AI Consultation (Symptom Chat)	Free-text symptoms are collected and structured with visible progress and confidence	Passed
Disease Prediction (KNN / Decision Tree / Logistic Regression)	Selected model executes inference and returns a primary diagnosis with confidence score	Passed
Differential Diagnosis Display	Ranked list of alternative conditions with individual confidence percentages is shown	Passed
Clinical Reasoning (Gemini)	Plain-language explanation of the predicted diagnosis is generated and displayed	Passed
Prediction History	Past predictions and consultations are stored and retrievable	Passed
Analytics Dashboard	Model usage, confidence trends, and symptom frequency charts render correctly	Passed
Responsive Layout	Application renders correctly across different browser window sizes	Passed

RESULTS

MediSense AI was successfully developed and tested as a single Streamlit application. All major modules performed as expected: patient and admin authentication, AI-driven symptom consultation, multi-model disease prediction (KNN, Decision Tree, and Logistic Regression), differential diagnosis with confidence scoring, and a live analytics dashboard. In a sample run, the Logistic Regression model correctly executed inference on user-reported symptoms and returned hypokalemia as the primary diagnosis with 37.9% confidence, alongside four ranked alternative conditions and a clear, LLM-generated clinical explanation — demonstrating that the prediction engine, the Gemini-based reasoning layer, and the analytics logging pipeline all worked together correctly end-to-end.

OUTPUT SCREENSHOT

Live diagnostic inference in progress on the Prediction Engine



CONCLUSION

The Disease Prediction Assistant, built as MediSense AI, successfully demonstrates a practical, explainable approach to AI-assisted symptom triage. By combining classical machine learning classifiers with a cloud large-language model inside a single Streamlit application, the platform allows users to describe symptoms naturally while still grounding every prediction in a trained, evaluable statistical model. Features such as differential diagnosis, specialist referral suggestions, consultation history, and a live analytics dashboard come together to form a coherent, end-to-end clinical-assistant experience, developed and deployed entirely as a solo project.

The successful implementation and testing of MediSense AI show that a lightweight, single-file Python stack — Streamlit, scikit-learn, and a cloud generative-AI API — is sufficient to build a genuinely useful healthcare-assistant tool, without the overhead of a separate backend service. Overall, the project highlights how classical ML and modern generative AI can be combined responsibly, with clear disclaimers and referral guidance, to support faster and better-informed health-related decisions.

FUTURE WORK

- Expanding the training dataset beyond 754 diseases and 377 symptoms with more recent, regionally relevant clinical data.
- Adding a fully offline local LLM option to remove dependence on the cloud Gemini API for privacy-sensitive deployments.
- Building a mobile-friendly or native app version for on-the-go symptom consultation.
- Supporting multilingual symptom input to make the assistant accessible to a wider population.
- Integrating with real Electronic Health Record (EHR) systems for doctor-verified, longitudinal patient history.
- Adding wearable-device integration to incorporate real-time vitals (heart rate, temperature, SpO2) into predictions.

REFERENCES

1. Streamlit Documentation. Available at: docs.streamlit.io/
2. scikit-learn Documentation. Available at: scikit-learn.org/stable/
3. Google AI — Gemini API Documentation. Available at: ai.google.dev/gemini-api/docs
4. Plotly Python Documentation. Available at: plotly.com/python/
5. Pandas Documentation. Available at: pandas.pydata.org/docs/
6. NumPy Documentation. Available at: numpy.org/doc/
7. Python Documentation. Available at: docs.python.org/3/
8. SQLite Documentation. Available at: sqlite.org/docs.html